

MedGuard AI

Real-time medical emergency monitoring using computer vision, a FastAPI backend, and a browser dashboard.

This repository is based on the project described in the research/demo material in `readme.pdf`: live video monitoring, YOLO-based detection, emergency alerts, and a central monitoring dashboard. Some older docs in the project still mention Docker, React/Next.js, and PostgreSQL. For this local folder, the working setup is different:

- Backend: FastAPI
- Frontend: static HTML dashboard in `frontend/dashboard`
- Local database: SQLite
- Camera source: webcam by default (`CAMERA_SOURCES=0`)
- Chat assistant: disabled by default for stable local startup

This README is the step-by-step guide for the current working project state on Windows and VS Code.

What The Project Does

MedGuard AI is designed to:

- Monitor a live camera feed
- Detect events such as falls, seizures, unconsciousness, tremor, facial distress, and cardiac-like distress patterns
- Show the live feed and event status in a dashboard
- Expose camera, health, and detection APIs for testing

Current Working Local Setup

Use this local workflow, not the old npm or Docker-only flow:

- Start the backend with `uvicorn`
- Serve the dashboard with Python's built-in HTTP server
- Open the dashboard in Chrome
- Test the camera with the `health`, `snapshot`, and `stream` endpoints

Important:

- Do not run `npm run dev` from the project root. There is no root `package.json`.
- The dashboard is a static HTML app, not a running React dev server in this folder.
- WS: `CONNECTED` and WS: `FALLBACK POLL` are both acceptable on the dashboard. `FALLBACK POLL` means the dashboard is using REST polling instead of a persistent WebSocket.

Project Structure

Key paths you will actually use:

- `backend/api/main.py` - FastAPI app entrypoint
- `frontend/dashboard/index.html` - dashboard UI
- `scripts/test_camera.py` - direct camera test utility
- `.env` - local configuration
- `data/medguard.db` - local SQLite database created automatically

First-Time Setup

If the venv folder already exists and the project has already been installed, you can skip to the next section.

If you need to set the project up from scratch on Windows:

- 1 Open PowerShell in `D:\medguard-ai`
- 2 Create a virtual environment:

```
python -m venv venv
```

- 1 Activate it:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned  
.\venv\Scripts\Activate.ps1
```

- 1 Install dependencies:

```
pip install --upgrade pip  
pip install -r requirements.txt
```

Optional Windows helper:

```
.\scripts\setup-windows.ps1
```

Local Configuration

The repository is already configured for a stable local run. These are the important defaults in `.env`:

- `DATABASE_URL=sqlite+aiosqlite:///./data/medguard.db`
- `CAMERA_SOURCES=0`
- `ENABLE_CHAT_ASSISTANT=false`
- `VECTOR_DB=faiss`

If you want to use a different camera later:

- CAMERA_SOURCES=0 means the default webcam
- You can replace it with another camera index such as 1
- You can also use an RTSP URL if needed

How To Run In VS Code

Open the folder D:\medguard-ai in VS Code, then use two terminals.

Terminal 1: Start The Backend

- 1 Open a new PowerShell terminal
- 2 Run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\env\Scripts\Activate.ps1
python -m uvicorn backend.api.main:app --host 0.0.0.0 --port 8000 --reload
```

- 1 Wait for these messages:

```
Application startup complete.
camera_connected
```

Expected behavior:

- The backend stays running in this terminal
- The webcam connects
- The local SQLite database is created automatically if needed

Terminal 2: Start The Frontend

- 1 Open a second PowerShell terminal in D:\medguard-ai
- 2 Run:

```
python -m http.server 3000 --directory frontend/dashboard
```

This serves the static dashboard at port 3000.

Open The Dashboard

Open this exact URL in Chrome:

```
http://127.0.0.1:3000/?v=8&apiHost=127.0.0.1&apiPort=8000
```

Then press:

```
Ctrl+F5
```

This forces a hard refresh so the browser does not keep an old cached dashboard.

How To Test The Camera

Use these tests in order.

1. Health Test

Run:

```
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:8000/health | Select-Object -ExpandProperty Content
```

Expected result:

- JSON output
- "status":"healthy"
- cam_0 appears under cameras
- running should be true

2. Snapshot Test

Run:

```
start http://127.0.0.1:8000/api/camera/cam_0/snapshot?ts=1
```

Expected result:

- A still image from your camera opens in the browser

3. Stream Test

Run:

```
start http://127.0.0.1:8000/api/camera/cam_0/stream
```

Expected result:

- A live MJPEG camera stream opens in the browser

4. Dashboard Test

Open:

```
http://127.0.0.1:3000/?v=8&apiHost=127.0.0.1&apiPort=8000
```

Expected dashboard state:

- SYSTEM ACTIVE

- ACTIVE CAMERAS shows 1
- CAM_0 shows LIVE
- The live tile for cam_0 displays the feed
- WS: CONNECTED or WS: FALLBACK POLL

Optional Direct Camera Test Script

The repo also includes a direct camera tester:

```
python scripts/test_camera.py --source 0 --no-models
```

Useful variants:

```
python scripts/test_camera.py --source 0 --debug
python scripts/test_camera.py --source 0 --cpu
python scripts/test_camera.py --source 0 --skip-inference
```

Notes:

- A window opens for the test
- Press Q to quit
- --no-models is the fastest way to verify the camera itself
- --skip-inference is useful if you want display only

Normal Dashboard Status Meanings

- SYSTEM ACTIVE - backend is healthy and at least one camera is live
- CAMERAS OFFLINE - no running camera was reported by /health
- WS: CONNECTED - dashboard is using WebSocket events
- WS: FALLBACK POLL - dashboard is using REST polling for events
- INFERENCE ACTIVE - the backend sees at least one live camera

Troubleshooting

Camera shows offline

Check:

```
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:8000/health | Select-Object -ExpandProperty Content
```

If cam_0 is not running:

- close Zoom, Teams, Meet, OBS, or any other app using the webcam
- restart the backend

- verify `.env` still has `CAMERA_SOURCES=0`

Snapshot works but dashboard is black

Try:

- 1 Close extra MedGuard tabs
- 2 Reopen the dashboard URL
- 3 Press `Ctrl+F5`

If the snapshot URL shows the picture, the backend camera path is working.

WebSocket keeps reconnecting

If the dashboard shows `WS: FALLBACK POLL`, the dashboard is still usable. This is not a blocker for local testing.

Chat warnings appear in backend logs

That is expected if chat is disabled or optional AI packages are not installed. The monitoring system still works.

Wrong old instructions

If any old note tells you to do one of these:

- `npm run dev`
- run a React app from the root
- require PostgreSQL just to test the local camera

Ignore those instructions for local testing in this repository. Use this README instead.

Recommended Test Order

Use this exact order for the smoothest local test:

- 1 Start backend
- 2 Confirm Application startup complete.
- 3 Run `/health`
- 4 Run `/api/camera/cam_0/snapshot`
- 5 Run `/api/camera/cam_0/stream`
- 6 Start frontend server
- 7 Open dashboard URL
- 8 Hard refresh with `Ctrl+F5`

Summary

For a normal local run, these are the only two commands you usually need:

Backend:

```
python -m uvicorn backend.api.main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend:

```
python -m http.server 3000 --directory frontend/dashboard
```

Then test:

- `http://127.0.0.1:8000/health`
- `http://127.0.0.1:8000/api/camera/cam_0/snapshot?ts=1`
- `http://127.0.0.1:8000/api/camera/cam_0/stream`
- `http://127.0.0.1:3000/?v=8&apiHost=127.0.0.1&apiPort=8000`